

Señales POSIX

- Introducción
- Versión original: Señales POSIX 1003.1a
- Señales POSIX 1003.1b (señales “de tiempo real”)
- Tratamiento de señales en procesos multihilo
- Inconvenientes y ventajas de las señales

Señales POSIX

- Introducción
 - Aparecen con UNIX como medio para notificar eventos (errores, terminación, tiempo...)
 - Evolución:
 - Normalizadas en POSIX 1003.1a (similares a señales UNIX)
 - Mejoradas en POSIX 1003.1b (“señales de tiempo real”)
 - Adaptadas a entornos multihilo (1003.1c)
 - Utilidad: Notificación de excepciones o eventos de diversos servicios POSIX, en particular los temporizadores

Versión original: Señales POSIX 1003.1a

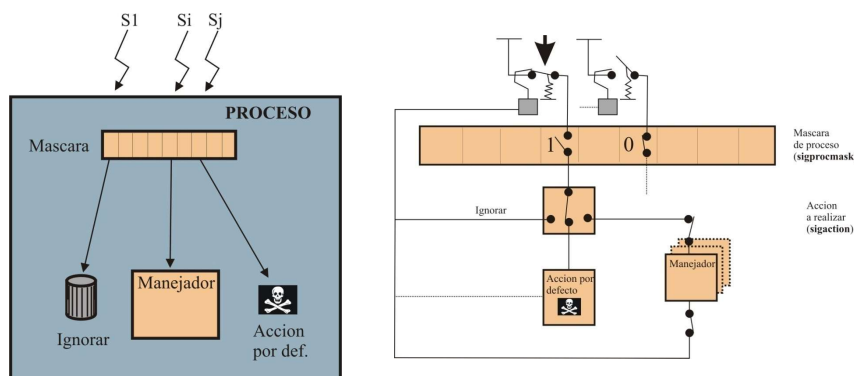
- Normalización de señales UNIX
 - No existía el concepto de hilo (o un proceso tenía sólo un “hilo”)
- En recepción, dos problemas a considerar:
 - Máscara de señales del proceso (luego máscara del hilo...)
 - Las señales pueden enmascarse (o bloquearse) individualmente
 - Las señales enmascaradas quedan pendientes (se memoriza su activación, pero no tienen efecto hasta ser desenmascaradas)
 - Acción a realizar:
 - Acción por defecto (habitualmente: abortar el proceso)
 - Ignorar (la señal no queda pendiente, esté o no enmascarada)
 - Ejecutar manejador (código excepcional que interrumpe temporalmente el proceso)
- Algunas señales:
 - SIGTERM, SIGKILL, SIGINT, SIGQUIT: Terminación de procesos
 - SIGFPE: Error aritmético
 - SIGALRM: Fin de temporizador
 - SIGUSR1, SIGUSR2: De propósito general (las únicas...)

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

3

Señales (1003.1a)



20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

4

Versión original: Señales POSIX 1003.1a

- **Modificar máscara:**

`int sigprocmask(int acción, sigset_t *iset, sigset_t *oset);`

- **acción:**

- SIG_BLOCK: Bloquear las señales seleccionadas (las demás se quedan como estaban)
- SIG_UNBLOCK: Desbloquear las señales seleccionadas (las demás se quedan como estaban)
- SIG_SETMASK: Cambiar toda la máscara (**todas** cambian)

- **iset:** Conjunto de señales seleccionadas o nueva máscara

- **oset:** Para obtener máscara anterior

- **Para selecc. señales o definir máscara, tipo sigset_t:**

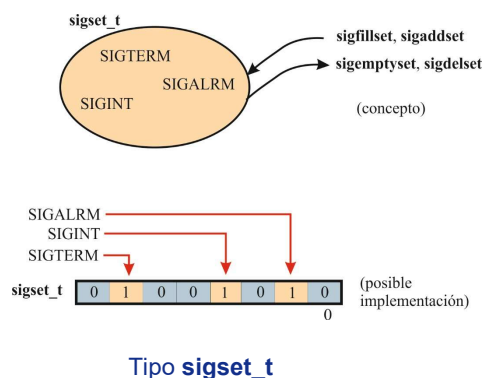
- Es un **conjunto de señales**, **no es** la máscara
- Vaciar: `int sigemptyset(sigset_t *pset);`
- Llenar con todas: `int sigfillset(sigset_t *pset);`
- Añadir **sig**: `int sigaddset(sigset_t *pset, int sig);`
- Eliminar **sig**: `int sigdelset(sigset_t *pset, int sig);`

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

5

Versión original: Señales POSIX 1003.1a



20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

6

Versión original: Señales POSIX 1003.1a

- Una señal enmascarada (o bloqueada)
 - No provoca ninguna acción si es enviada al proceso pero...
 - Se queda “pendiente”, salvo si la acción es “ignorar”
 - Cuando la señal pendiente se desenmascara, se produce la acción programada
 - En la versión original del sistema de señales la activación de una señal que ya está pendiente **se pierde**
- Otras llamadas relacionadas con la máscara:
 - **sigpending** almacena en un **sigset_t** las señales pendientes:

```
int sigpending(sigset_t *iset);
```

- **sigismember** permite saber si una señal está en un conjunto:

```
int sigismember(sigset_t *iset, int sig);
```

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

7

Versión original: Señales POSIX 1003.1a

- Cambiar acción a realizar:


```
int sigaction(int sig, struct sigaction *iacc, struct sigaction *oacc);
```

 - **sig**: Señal a modificar
 - **iacc**: Acción a programar (si no es NULL):
 - **sa_handler**: SIG_IGN (ignorar), SIG_DFL (acción por defecto) o puntero al manejador
 - **sa_flags**: Flags; normalmente 0
 - **sa_mask**: Señales a añadir a la máscara del manejador
 - **oacc**: Acción anteriormente programada (si no es NULL)
- Máscara del manejador:
 - Señales ya enmascaradas en el proceso (luego hilo...)
 - Señal que lo ha activado
 - Señales presentes en **sa_mask**
- Dos señales pueden tener el mismo manejador

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

8

Versión original: Señales POSIX 1003.1a

- **Manejador:**
 - Función con argumento entero que no devuelve nada
 - El argumento indica el número de la señal
- **Llamadas “async-signal-safe”:**
 - Concepto más restrictivo que “thread-safe”
 - Llamadas “async-signal-safe” (o seguras en manejador): pueden ser invocadas en un manejador **sin restricciones**
 - Existe una lista de funciones que son seguras: **fork**, **read**, ...
 - Otras no tienen por qué serlo (**printf**, casi todas las **pthread_XXX**, ...)
 - Una llamada no segura en manejador podrá dar problemas sólo **si el manejador interrumpe otra llamada no segura**
- **¿Por qué puede no ser segura una llamada?**
 - Habitualmente, debido al uso de datos globales
 - No puede evitarse con controles de acceso
 - Puede evitarse enmascarando señales mientras se ejecutan

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

9

Versión original: Señales POSIX 1003.1a

- **Tratamiento asíncrono:**
 - Es el mecanismo previsto originalmente
 - Se programa un manejador como acción a realizar
 - Al recibirse la señal, el manejador se ejecuta, interrumpiendo el proceso temporalmente
 - Posibles problemas con llamadas no “async-signal-safe”
 - Interrumpe diversas llamadas que implican espera (error EINTR) como **sleep**, **wait** u operaciones de entrada/salida
- **Tratamiento síncrono:**
 - El proceso espera a la llegada de la señal (no tiene nada que hacer hasta ese momento...)
 - No estaba previsto: Puede hacerse utilizando la interrupción de esperas por error EINTR (una especie de “truco”)
 - Siempre ha de ejecutarse el manejador (aunque no haga nada)
- **Enviar señales:**

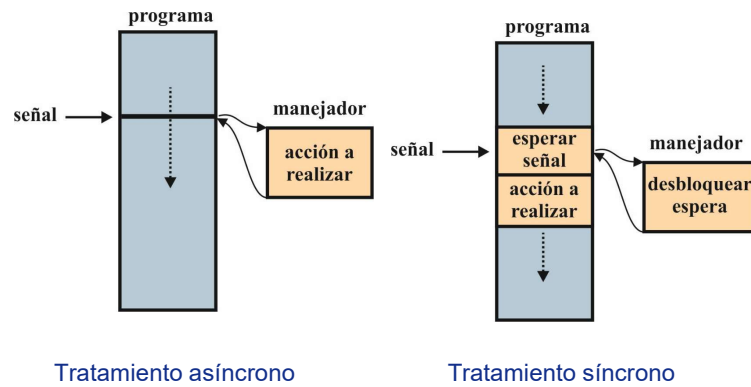
`int kill(pid_t destino, int señal);`

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

10

Versión original: Señales POSIX 1003.1a



20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

11

Señales POSIX 1003.1a: Tratamiento asíncrono

Ejemplo: Manejador para SIGTERM y SIGINT

```
(...)
#include <signal.h>
(...)
void manej(int s) { .... }
(...)
struct sigaction acc;
sigset_t sen;

/* Programar manejador */
acc.sa_flags = 0;
acc.sa_handler = manej;
sigemptyset(&acc.sa_mask);
sigaction(SIGTERM, &acc, NULL);
sigaction(SIGINT, &acc, NULL);

/* Desenmascarar ambas señales */
/* Antes hay que hacer un conjunto con
   SIGTERM y SIGINT */
sigemptyset(&sen);
sigaddset(&sen, SIGTERM);
sigaddset(&sen, SIGINT);

/* Cambiando sen no cambiamos la
   máscara; lo haremos ahora... */
sigprocmask(SIG_UNBLOCK, &sen, NULL);
/* A partir de ahora se recibirán señales y
   entrará manej asincrónicamente */
(...)
```

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

12

Señales POSIX 1003.1a: Tratamiento síncrono

Ejemplo: Esperar SIGUSR1

```
(...)
#include <signal.h>
(...)
/* Incluso ahora necesitamos
   manejador */
void manej(int s) { .... }
(...)
struct sigaction acc;
sigset_t sen;
int res;

/* Programar manejador */
acc.sa_flags = 0;
acc.sa_handler = manej;
sigemptyset(&acc.sa_mask);
sigaction(SIGUSR1, &acc, NULL);

/* Desenmascarar SIGUSR1 */
sigemptyset(&sen);
sigaddset(&sen, SIGUSR1);
sigprocmask(SIG_UNBLOCK, &sen, NULL);
/* Es necesario que entre manej para romper
   la espera */
(...)
/* Esperar SIGUSR1 (sleep devuelve tiempo
   restante) */
res = sleep(3600);
if(res != 0) {
    printf("Ha llegado SIGUSR1\n");
    hacer_cuando_SIGUSR1(...);
} else {
    printf("Error: No llega SIGUSR1!!\n");
    hacer_trat_error(...);
}
(...)
```

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

13

Ejemplo: Sobretiempo con señales

```
#include <unistd.h>
#include <signal.h>
int timeout; /* Flag de sobretiempo */
/* Manejador de SIGALRM */
void maneja(int sign) {
    timeout = 1;
}
void main(void) {
    int fin; /* Flag de fin */
    struct sigaction action;
    sigset_t set;

    /* Programación de la acción a realizar */
    action.flags = 0;
    action.sa_handler = maneja;
    sigemptyset(&action.sa_mask);
    sigaction(SIGALRM, &action, NULL);

    /* Desbloqueo de SIGALRM */
    sigemptyset(&set); sigaddset(&set,
    SIGALRM);
    sigprocmask(SIG_UNBLOCK, &set, NULL);

    /* Después se programa el "despertador" */
    timeout = 0;
    alarm(20);
    fin = 0;

    /* Algoritmo con sobretiempo */
    while(!fin && !timeout) {
        fin = funcion_iteracion(...);
    }

    /* Analizar lo que ha sucedido */
    if(fin) alarm(0); /* Ya no debe saltar */
    if(timeout) {
        printf("Alarma! Sobretiempo!\n");
        < algoritmo alternativo >
    }
}
```

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

14

Señales POSIX 1003.1b (señales “de tiempo real”)

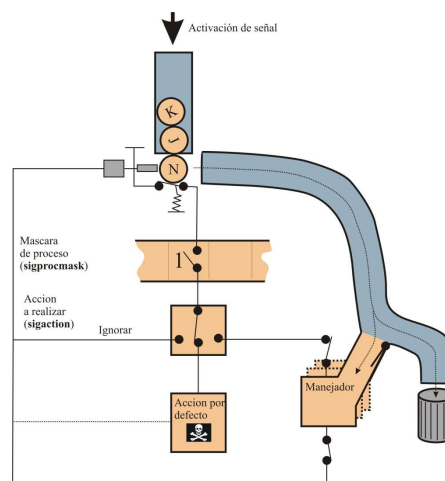
- **Problemas de POSIX 1003.1a:**
 - Pocas señales de propósito general
 - No se encolan (pueden perderse señales)
 - No se conoce el orden de recepción
 - No pueden transmitir datos adicionales
 - Son asíncronas (manejador imprescindible)
- **Mejoras de POSIX 1003.1b:**
 - Nuevas señales, de SIGRTMIN a SIGRTMAX
 - Encolamiento de señales pendientes
 - Dato adicional (entero o puntero)
 - Prioridad: Decreciente de SIGRTMIN a SIGRTMAX
 - Método de espera síncrona alternativo al manejador
- **Pero aún no existía el concepto de hilo...**

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

15

Señales POSIX 1003.1b



20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

16

Señales POSIX 1003.1b

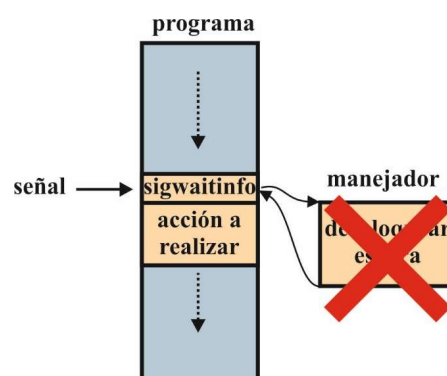
- Para disponer de las nuevas funciones (encolamiento, dato, prioridad):
 - Programar el flag SA_SIGINFO (con **sigaction**)
 - Usar nuevo manejador (**sa_sigaction**) en lugar del antiguo (**sa_handler**)
 - Sólo llega dato si se ha enviado (imposible con **kill**): Nueva llamada **sigqueue**
- Tratamiento síncrono sin manejador:
 - Llamadas **sigwaitinfo** y **sigtimedwait**
 - Hay que **enmascarar** las señales que se esperan, para cerrar el paso al manejador o la acción por defecto
 - La acción a realizar no puede ser SIG_IGN, o se perderá la señal
 - Pueden desbloquearse por error EINTR por otras señales que **no se esperan** y activan manejadores (solución: reintentar)
- Envío de señales:
 - Puede usarse **kill**, pero entonces no hay dato válido
 - Otra opción:
int **sigqueue**(pid_t destino, int señal, union sigval dato);

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

17

Señales POSIX 1003.1b



Tratamiento síncrono

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

18

Señales POSIX 1003.1b

- **Dato enviado con la señal:**

- Enviado por **sigqueue** y algunos servicios POSIX (temporizadores, colas de mensajes)
- Entero o puntero, en forma de unión:


```
union sigval {
    int sival_int;
    void *sival_ptr;
}
union sigval us; /* Definir union sigval */
us.sival_int = 7; /* Inicializar como entero */
```

- **Datos recibidos con la señal (**siginfo_t**):**

- Recibido por manejador, **sigwaitinfo**, **sigtimedwait**
- Es un **siginfo_t**; es decir, el dato (**union sigval**) más datos auxiliares:


```
typedef struct {
    int si_signo; /* Número de la señal */
    int si_code; /* Causa de la señal */
    union sigval si_value; /* Dato enviado con la señal */
    int drec;
} siginfo_t info;
/* Definir siginfo_t */
/* Copiar dato como entero */
drec = info.si_value.sival_int;
```

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

19

Señales POSIX 1003.1b: Tratamiento asíncrono

Ejemplo: Manejador que recibe datos

```
(...)
#include <signal.h>
(...)
Int n = 0; /* Suma de datos */
/* Manejador con nueva interfaz */
void mtr(int s, siginfo_t *e, void *p) {
    n = n + e->si_value.sival_int;
}
(...)
struct sigaction acc;
sigset_t sen;

/* Programar manejador */
acc.sa_flags = SA_SIGINFO;
acc.sa_sigaction = mtr;
sigemptyset(&acc.sa_mask);
sigaction(SIGRTMIN, &acc, NULL);

/* Desenmascarar SIGRTMIN */
sigemptyset(&sen);
sigaddset(&sen, SIGRTMIN);
sigprocmask(SIG_UNBLOCK, &sen, NULL);

/* A partir de ahora mtr entrará
asíncronamente y sumará en n los datos
asociados a la señal */
(...)
```

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

20

Señales POSIX 1003.1b: Tratamiento síncrono

Ejemplo: Recepción de datos con **sigwaitinfo**

```
(...)
#include <signal.h>
(...)
Int n = 0; /* Suma */
/* Manejador (nunca entra) */
void mtr(int s, siginfo_t *e, void *p) { }
(...)
struct sigaction acc;
sigset_t sen; siginfo_t info;

/* Programar manejador */
/* También valdría SIG_DFL */
acc.sa_flags = SA_SIGINFO;
acc.sa_sigaction = mtr;
sigemptyset(&acc.sa_mask);
sigaction(SIGRTMIN, &acc, NULL);

/* Enmascarar SIGRTMIN */
/* Si no, no hay garantía de que funcione */
sigemptyset(&sen);
sigaddset(&sen, SIGRTMIN);
sigprocmask(SIG_BLOCK, &sen, NULL);

/* Bucle para sumar los datos */
/* mtr nunca entra */
while(1) {
    sigwaitinfo(&sen, &info);
    n = n + info.si_value.sival_int;
}
(...)
```

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

21

Señales POSIX 1003.1b: campo **si_code**

- Permite conocer la causa de la señal
- Opciones:
 - SI_QUEUE: Enviada por **sigqueue**; hay dato
 - SI_USER: Enviada por **kill**; no hay dato
 - SI_TIMER: Enviada por temporizador
 - Otras causas; por ejemplo, el tipo de error que ha provocado la señal, si es el caso

```
(...)
sigset_t s; siginfo_t info;
/* Bloquear señal */
sigemptyset(&s);
sigaddset(&s, SIGRTMIN);
sigprocmask(SIG_BLOCK, &s, NULL);
(...)
/* Esperar señal; suponemos SA_SIGINFO
activado */
sigwaitinfo(&s, &info);
if(info.si_code == SI_QUEUE)
    printf("Dato: %d\n", info.si_value.sival_int);
else
    /* Si se usó kill para enviar el dato no es
    válido */
    if(info.si_code == SI_USER)
        printf("No hay dato; enviada por kill\n");
    else printf("Causa no esperada\n");
(...)
```

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

22

Señales en procesos multihilo

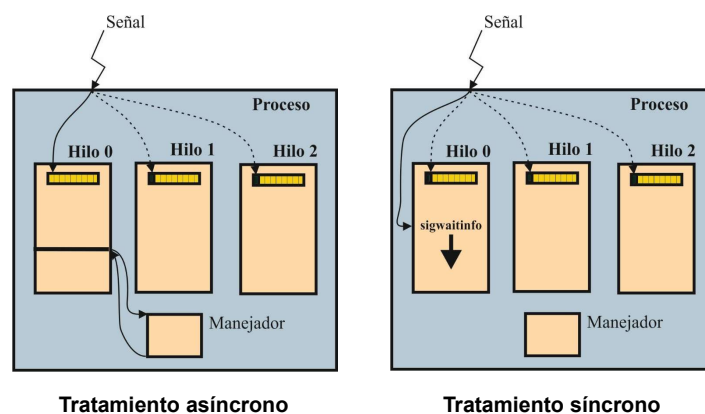
- Al introducir los hilos, ¿Quién se ve afectado la señal?
- Selección del hilo destinatario:
 - Tres casos:
 - Señal generada síncronamente por error: El hilo que la provoca
 - Señal enviada a un hilo con **pthread_kill**: Destinatario perfectamente definido
 - Señal enviada al proceso completo: Caso a discutir
 - Cada hilo tiene una máscara propia que se modifica con **pthread_sigmask**:
 - Tratamiento asíncrono (con manejador):
 - El manejador es común para todos los hilos
 - Si la señal está enmascarada en todos menos uno, el manejador se ejecuta en el contexto de ese hilo
 - En caso contrario, no se puede predecir el destinatario
 - Tratamiento síncrono (**sigwaitinfo** y otras):
 - Señal enmascarada en todos los hilos
 - Sólo un hilo esperando a la vez la señal, si no, destinatario indefinido

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

23

Señales en procesos multihilo



20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

24

Señales en procesos multihilo

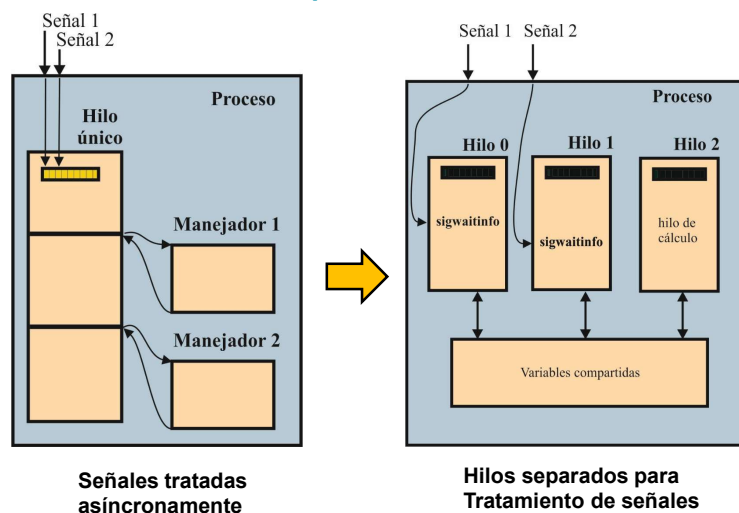
- Particularidades de las funciones **pthread_xx**:
 - No se exige que la mayoría sea “async-signal-safe”:
Posibles problemas para usarlas en un manejador
 - Excepciones: **pthread_kill**, **pthread_self**, **pthread_sigmask** sí deben ser seguras en un manejador
 - No se rompen las esperas cuando salta un manejador (no se genera el error EINTR)
 - Por ejemplo, **pthread_mutex_lock** o **pthread_join** no dejan de esperar porque se ejecute un manejador
- Algunas conclusiones:
 - Parece aconsejable en muchos casos tratar las señales **síncronamente**, dedicando hilos a esperar señales
 - Si se enmascaran las señales en **main**, el resto de los hilos heredará la máscara
 - Tiene sentido tratar asíncronamente señales en un hilo **para romper la espera** de llamadas 1003.1a o 1003.1b (por ejemplo, para sobretiempo)

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

25

Señales en procesos multihilo



20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

26

Señales: Ventajas e inconvenientes

- **Inconvenientes:**
 - Tratamiento poco eficiente (sobre todo si es asíncrono)
 - Naturaleza asíncrona (salvo señales de 1003.1b)
 - Poco ancho de banda para transmitir datos
- **Ventajas:**
 - Son asíncronas (si esa funcionalidad es la que se necesita; con hilos, raramente)
 - Relativamente simples de programar
 - Orientadas a un proceso en particular (cualidad poco habitual)
 - No suelen verse afectadas por restricciones y se dispone siempre de ellas

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

27

Señales: Puntos a recordar

- **Máscara de señales**
 - Cada hilo tiene su máscara
 - Lo que antes era “máscara del proceso” es ahora la máscara del único hilo que tiene un proceso sin múltiples hilos
 - El hilo **main** no es “el proceso” – es un hilo, con su máscara
 - La máscara inicial se hereda de
 - El hilo que ejecuta **pthread_create** (hilo “padre”, por así decir), o bien...
 - El hilo que ejecutó **fork** o **execl** en el proceso padre (para el caso del primer hilo de un proceso – **main**, normalmente)
 - Si llega una señal y está enmascarada, **queda pendiente**
 - Para señales SIGRTMIN+n con flag SA_SIGINFO, se encola
 - En otro caso, puede que se pierdan posteriores activaciones
- **Acción a realizar**
 - Manejador, acción por defecto (SIG_DFL) o ignorar (SIG_IGN)
 - No hay garantías de poder usar SA_SIGINFO salvo con señales SIGRTMIN+n
 - La acción a realizar es compartida por todos los hilos

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

28

Señales: Puntos a recordar

- ¿Quién recibe una señal?
 - Una señal dirigida a un proceso (temporizador, **kill**, **sigqueue**...) sólo es recibida por uno (y sólo uno) de los hilos de su destinatario
 - Si hay varios hilos, **no tiene sentido decir que la reciba “el proceso”**, como si fuese una entidad distinta de los hilos
 - Los procesos hijos del proceso destinatario no reciben la señal en ningún caso
 - Desde de un proceso puede enviarse una señal (sin dato) a uno de sus hilos (**pthread_kill**)
- ¿Qué hilo recibe la señal?
 - Si no tenemos cuidado, **cualquiera** (salvo con **pthread_kill**)
 - A menos que nos de igual, hay que dejar sólo un camino a la señal:
 - En tratamiento síncrono, bloquearla en **todos** (no sólo en el que ha de recibirla) y esperarla sólo en uno
 - En tratamiento asíncrono, programar manejador y bloquearla en **todos salvo uno**

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

29

Señales: Puntos a recordar

- Programación de los flags
 - Para conservar la portabilidad, sólo se debería utilizar SA_SIGINFO con las señales de tiempo real (SIGRTMIN...)
 - Interesa usar SA_SIGINFO cuando
 - Necesito un manejador “de tiempo real” (**sa_sigaction**)
 - O bien quiero asegurarme de que funcionan las mejoras de la norma 1003.1b (encolamiento, dato adicional, prioridad...)
- Programación de la acción a realizar
 - Si la señal se trata asincrónamente, programar un manejador (**sa_handler** o **sa_sigaction**, según corresponda)
 - Si la señal se trata síncronamente, conviene asegurarse de que la acción no es SIG_IGN, aparte de enmascararla
- Prog. de máscara adicional del manejador (**sa_mask**)
 - Vacía, si no es problema que salten otros manejadores dentro del que estamos programando
 - Con las señales cuyo manejador no debe saltar dentro del que estamos programando, si es el caso
 - Llena, si no queremos que el manejador se interrumpa nunca

20/10/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

30